

CS383 - Software Engineering

Software Requirements

Requirements overview

- In real life, you need software if
 - Essentially needed
 - Economically justified
- Software requirement: what is needed or wanted from software?
- Requirements can range
 - abstract single statement that may be need interpretation and expansion
 - mathematical functions and notations

Requirements types

1. User requirement

- Taking from users' statements of natural language
- Diagrams can be used to represent components and services
- High-level abstract requirements
- Services/functionalities that system is expected to provide
- Conditions/constraints must be addressed when services are provided

1. System requirement

- A more detailed description of the system's functions, services and constraints
- SRS must document user and system requirements
- SRS describes what is to be developed (design and implemented)

Requirements types and people involved

- Requirements should be written at different level of details (why?)
 - People who are interested in reading them have different background

- User requirements reader
 - Not usually interested about how the system should be implemented

User Requirements

Client Managers
System End-Users
Client Engineers
Contractor Managers
System Architects

- System requirements reader
 - Want to know exactly what the system will do
 - How it will support business processes
 - Or they are involved in the system development/implementation

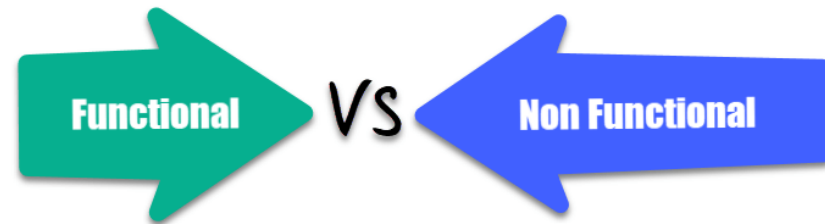
System Requirements

System End-Users
Client Engineers
System Architects
Software Developers

Requirement classification

1. Functional requirements

- Services and functions that the system should provide
- Sometimes, it may include services/actions that the system should not do



1. Non-functional requirements

- Applied to system as whole (all functions)
- Constraints on the system including
 - Product requirements (e.g. timing, efficiency, performance)
 - Organization requirements (e.g. development, environmental)
 - External requirements (e.g., standards/ethical requirements)

Function requirements

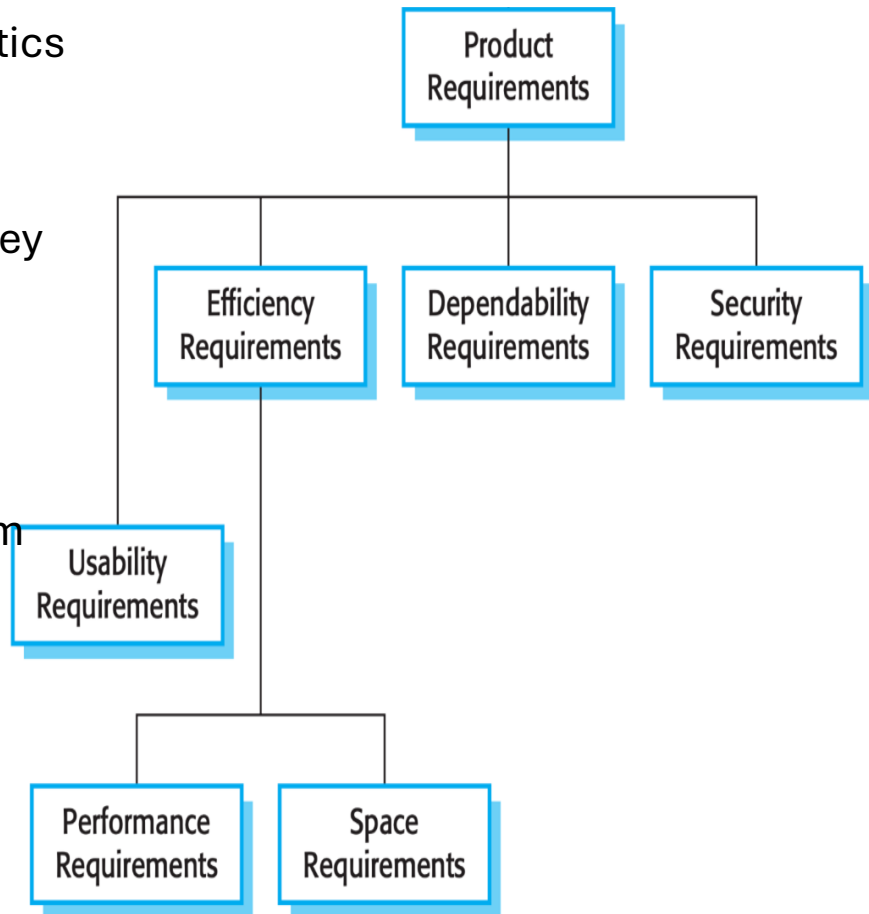
- Requirement can be
 - Vague, unreasonable, contradictory, simply impossible
 - Clients may not present what they want technically (remember abstract)
 - SW engineers ought to have meetings to clarify what the clients really need
 - They may clarify with their managers instead of clients
 - Requirements can be ambiguous/overlaps if interpreted by different people of different background
- Requirements need to be complete and consistent when documented
 - In reality, it is sometimes hard to produce complete and consistency document of requirements

Non-functional requirements

- Define system properties/characteristics and constraints
- They may be more crucial than functional requirements
 - The system may not operate if they are absent

1. Product requirements

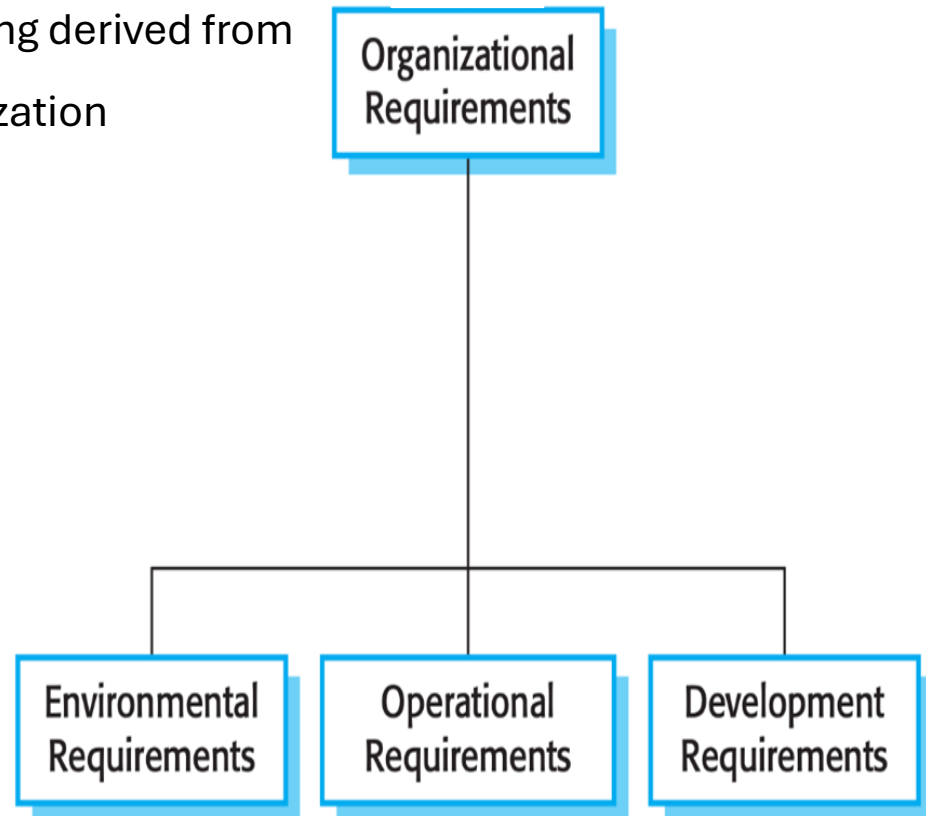
- Specify the constraints on system behaviour



Non-functional requirements (cont.)

2. Organization requirements:

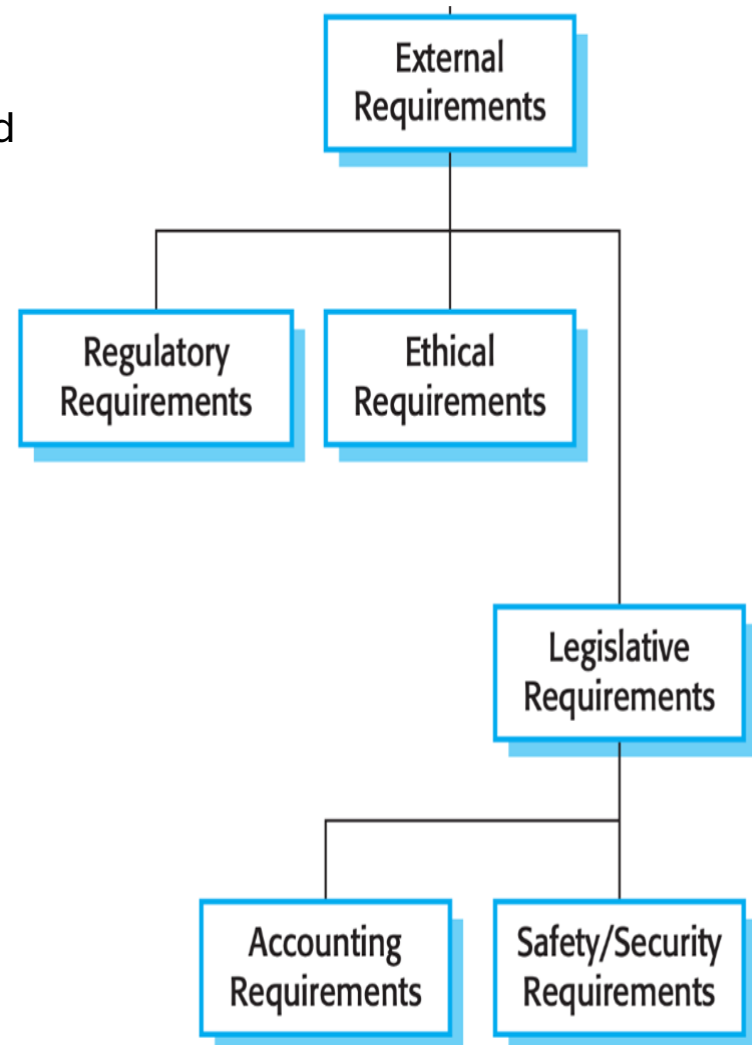
- Related to policies and procedures involved in system/software engineering derived from customer/developer's organization



Non-functional requirements (cont.)

3. External requirements:

- Related to regulations, legalization and ethical requirements



Metrics needed for non-functional requirements

- Speed
 - Processed transactions, responsiveness of the service, refresh time
- Size
 - Mbytes and number of storage devices
- Ease of use
 - Training time, number of elements on page
- Reliability
 - Mean time to fail
 - Number of failure occurrences
- Robustness
 - Time to restart after failure
 - Percentage of data lost on failure
- Portability
 - Number of target systems

Examples of requirements

- **Example 1: a university public computer**

Public access computers are located across university campuses, and they have access to the internet. The computer should allow users to access these computers and provide a decent and acceptable internet connection. Each user is allowed to access the computer for 10 minutes from the start of the session. At the end of the session, the user is allowed to renew the session for another 10 minutes or exit. Access renewals should only be allowed three times in a row. The content to which the user can access should not violate regulations put forward by the government.

1. What are the functional requirements?
2. What are the non-functional requirements

Examples of requirements

- **Example 2: a dynamic traffic light**

Dynamic traffic lights operate automatically in a city. They are fitted with sensors that detect any movement toward them. If there is movement, the traffic is activated and should turn green once its turn comes up. If there is no movement, the traffic continues to be inactive. The amount of time allowed for the traffic to be green is proportional to the number of vehicles detected by the sensors but should be at most 90 seconds. Also, if no movement is detected, the traffic light turns red. Finally, the traffic light should address nightlight efficiency to save power.

1. What are the functional requirements?
2. What are the non-functional requirements

Examples of requirements

- **Example 3: a patient management system**

A hospital is asking you to develop a system for managing its patient records. The system shall allow the creation of new records for only newly registered patients using national IDs. Also, it shall allow retrieving patient information using IDs, names, or phone numbers. The system should be operational under the Java Virtual Machine using JDK 17 and above.

For the hospital convenience, the system should quickly response to operations (ideally less than 1 second). Also, the patients information should be end-to-end encrypted for security purposes.

1. What are the functional requirements?
2. What are the non-functional requirements

SRS Documentation

- Requirement specifications are the process of documenting user and system requirements in detail.
 - Including functional and non-functional requirements
- Methods of writing requirement specifications:
 1. Natural language sentences: each sentence should states one requirement
 2. Structured natural language: like previous one but according to agreeable format (mostly used)
 3. Design description language: using programming language but with abstract features and defining operation of a feature of system (rarely used)
 4. Graphical notations: using models annotations to define functional requirements (e.g. using UML)
 5. Mathematical specification: using mathematical concepts such as finite-state automata, set notations and algebra

SRS Documentation (cont.d)

- Use a standard format for all requirements
 - At least, a requirement should have:
 - Description
 - Rationale statement (why it is needed,)
 - Inputs and their source
 - Outputs and their destination
 - Actions to be taking for the requirement, pre-conditions, post-conditions

SRS Documentation (cont.d)

- Use text highlight for keywords or emphasis for important sentences.
- Use consistent language for describing requirements
 - Use shall for mandatory
 - Use should for desirable
- Have precision and clarity when documenting a requirement
- **There is an outline for SRS document that should be followed**

Functional Requirement Documentation example

Insulin Pump/Control Software/SRS/3.3.2

Function	Compute insulin dose: Safe sugar level.
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1).
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose—the dose in insulin to be delivered.
Destination	Main control loop.
Action	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requirements	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin.
Post-condition	r0 is replaced by r1 then r1 is replaced by r2.
Side effects	None.

SRS Documentation Outline

1. Introduction
 1. Purpose
 2. Product scope
 3. References
 4. Structure
2. System Overview
 1. Product perspective
 2. Product features
 3. User roles and characteristics
 4. Operating environment
 5. Design and implementation constraints
 6. Assumptions and dependencies
3. Requirements Engineering
 1. Requirements elicitation
 2. Requirements analysis
 3. Requirements validation
4. Functional Requirements
 1. Requirement 1
 2. Requirement 2
 3. ...
4. Non-functional Requirements
 1. Performance Requirements
 2. Safety Requirements
 3. Security Requirements
 4. Software Quality Attributes
5. External Interface Requirements
 1. User Interface requirements
 2. Hardware Interface requirements
 3. Software Interface requirements
 4. Communication Interface requirements

Requirements management

- Changing or evolving of requirements is inevitable (very certain). Reason can be:
 - Business and technical environment of the system changed
 - New hardware, new interface is needed, new legislation and regulation
 - The customers (people who pay for the system) may impose new requirements because of organizational or budgetary constraints
 - This affect the end users of the system
 - The customers and end users are not always the same

Requirements management tools

- Doing requirements change management manually can be an impossible for big systems
- Management tools should allow:
 1. Requirement storage: requirements should be securely centralized in a particular storage
 - Centralization of data is key element for planning, and decision making activities
 - Access should be given to all those involving in requirement engineering process

Requirements management tools

- Management tools should allow:

2. Change management: taking as input an identified problem, then

- Analyze the problem and change specification
- Change analysis and costing
- Change implementation

Requirements management tools

- Management tools should allow:

3. Traceability management

- Tools of traceability allow identifying of related requirements that affected by a change